

Software Requirements Specification

Gema Event Management Platform

Document Version: 1.0

Date: 2026-02-25

Status: Draft

Prepared By: Engineering Team

Standard: IEEE 830-1998

Table of Contents

1. [Introduction](#)
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
 2. [Overall Description](#)
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
 3. [Functional Requirements](#)
 - 3.1 Authentication and Authorization
 - 3.2 Event Management
 - 3.3 Booking and Order Management
 - 3.4 Payment Processing
 - 3.5 Ticket Management
 - 3.6 Vendor Management
 - 3.7 Teacher Management
 - 3.8 Admin Management
 - 3.9 Commission and Revenue
 - 3.10 Content and Marketing
 4. [Non-Functional Requirements](#)
 - 4.1 Performance
 - 4.2 Security
 - 4.3 Scalability
 - 4.4 Reliability and Availability
 - 4.5 Maintainability
 - 4.6 Usability
 5. [External Interface Requirements](#)
 - 5.1 Stripe
 - 5.2 Firebase
 - 5.3 Cloudinary
 - 5.4 Redis
 - 5.5 BullMQ
 - 5.6 Nodemailer / SMTP
 6. [System Constraints](#)
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Gema Event Management Platform**. It is intended for use by the engineering team, product stakeholders, and QA personnel to guide development, testing, and future maintenance.

The platform enables vendors and teachers to list, manage, and monetize events, courses, workshops, and competitions. Customers discover and book these offerings. A platform administrator oversees all activity, moderates content, and manages financial operations including commissions and payouts.

1.2 Scope

The Gema platform is a multi-tenant, multi-role event management and ticketing system built for the UAE market with international payment support. The system comprises:

- A **backend REST API** (Node.js 20, Express 4.18, TypeScript 5.3) serving all business logic
- A **frontend single-page application** (React 18, Vite 4.5) consumed by all user roles
- A **background worker process** (BullMQ 5.60) handling email, QR code generation, analytics, and notifications

The system is **out of scope** for:

- Native mobile applications (iOS/Android)
- Real-time video/live-streaming for online events
- Point-of-sale (POS) hardware integrations

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Admin	Platform superuser with full system access
Vendor	Business entity listing events and courses on the platform
Teacher	Individual educator listing courses and classes
Customer	End-user who discovers and books events
Employee	Staff member assigned by a vendor to manage check-ins
Event	Any bookable offering including competitions, courses, workshops, classes, venues, olympiads, and masterclasses
Order	A single purchase transaction containing one or more event bookings
Ticket	A QR-code-enabled proof of purchase for a specific event attendee
Payout	A financial disbursement from the platform to a vendor or teacher
Commission	The platform's revenue share retained from each transaction
Stripe Connect	Stripe's product for routing payments to connected third-party accounts
AED	UAE Dirham — the platform's primary base currency
BullMQ	Redis-backed message queue for background job processing
KVM1	Single-core VPS deployment environment
JWT	JSON Web Token — the primary authentication token format
RBAC	Role-Based Access Control
2FA	Two-Factor Authentication
OTP	One-Time Password
PWA	Progressive Web Application
SRS	Software Requirements Specification

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Project CLAUDE.md: Gema codebase standards and architectural guidelines
- Stripe API Documentation: <https://stripe.com/docs/api>
- Firebase Admin SDK Documentation: <https://firebase.google.com/docs/admin/setup>
- Cloudinary API Documentation: <https://cloudinary.com/documentation>

1.5 Overview

Section 2 describes the system from a high level, covering its architecture, user classes, and operating constraints. Section 3 defines granular functional requirements organized by domain. Section 4 defines non-functional quality attributes. Section 5 describes external integrations. Section 6 lists hard system constraints.

2. Overall Description

2.1 Product Perspective

Gema is an independent, self-contained SaaS platform operating as the primary marketplace for activity and event discovery in the UAE and surrounding regions. It is not a component of a larger existing system.

The system follows a **layered REST API architecture**:

```
Client (React SPA / Mobile Browser)
|
HTTPS REST API (Express.js)
|
Service Layer (Business Logic)
|
Data Access (Mongoose ODM)
|
MongoDB 8.0 ↔ Redis (Cache + Queue)
|
External Services:
Stripe | Firebase | Cloudinary | Nodemailer
```

Background processing is handled by a separately-deployed BullMQ worker process that shares the same Redis instance. The frontend is a PWA supporting offline caching of static assets and network-first API responses.

2.2 Product Functions

The platform provides the following high-level capabilities:

1. Multi-role user registration, authentication, and profile management
2. Event, course, competition, and venue listing with rich scheduling, media, and SEO metadata
3. Cart-based and single-step booking with multi-currency display
4. Stripe-powered payment processing with platform and vendor Stripe Connect routing
5. Automated QR-code ticket generation and email delivery
6. QR-code check-in via mobile or employee portal
7. Vendor and teacher onboarding with verification workflow and document upload
8. Tiered commission engine with configurable rules per vendor or category
9. Payout request and disbursement lifecycle with admin approval
10. Admin dashboard with revenue analytics, moderation queue, and bulk operations
11. Affiliate link tracking, coupon discounts, and newsletter subscription
12. Blog, SEO content, homepage banners, announcement bars, and popup notifications
13. Cancellation, refund, and dispute management

2.3 User Classes and Characteristics

2.3.1 Administrator (Admin)

- Single or small team of platform operators
- Full system access: user management, event moderation, financial operations, settings
- Technical literacy expected; accesses dedicated `/admin/*` API routes
- Interacts primarily via an internal admin panel in the frontend

2.3.2 Vendor

- Business entity (training center, activity provider, venue operator) registered on the platform
- Creates and manages their own events, schedules, and pricing
- Manages bookings and attendees for their events
- Requests payouts; receives commission deductions on each sale (or pays a monthly subscription)
- Accesses `/vendors/*` and `/vendor/*` API routes
- May onboard employee accounts for check-in staff

2.3.3 Teacher

- Individual educator (freelance or employed) listed independently from vendors
- Creates and manages educational courses and classes
- Has a separate payment mode and subscription model from vendors
- Accesses `/teachers/*` API routes
- Linked to events via `teacherId` on the Event model

2.3.4 Customer

- End-user discovering, favoriting, and booking events
- Authenticates via email/password or social login (Google, Facebook, Apple)
- Receives QR-code tickets via email and can view orders in a self-service portal

- Accesses public event, search, and booking routes; authenticated order routes

2.3.5 Employee

- Check-in staff assigned by a vendor to one or more events
- Limited access: scan tickets at check-in gates using a dedicated check-in portal
- Accesses /checkin/* and /employees/* routes

2.4 Operating Environment

Component	Technology
Backend Runtime	Node.js 20.x on a KVM1 single-core VPS
Backend Framework	Express.js 4.18 + TypeScript 5.3
Database	MongoDB 8.0 (Mongoose ODM)
Cache & Queue Broker	Redis (ioredis)
Frontend Runtime	Node.js 22.x build; served as static assets via CDN/Vercel
Frontend Framework	React 18 + Vite 4.5
Operating System	Linux (production), Windows 11 (development)
Primary Region	UAE (Asia/Dubai timezone, AED base currency)

2.5 Design and Implementation Constraints

- **C-001:** The backend strictly follows a service-layer pattern; controllers handle HTTP I/O only — all business logic resides in service classes.
- **C-002:** TypeScript strict mode is disabled on the backend; type hints are provided where practical.
- **C-003:** Rate limiting is tuned for a KVM1 single-core server: 100 req/min general, 20 req/min for analytics/dashboard endpoints, 10 auth attempts per 15 minutes.
- **C-004:** JWT clock tolerance is fixed at 60 seconds to accommodate server time-sync variance.
- **C-005:** The charged currency is always INR or AED depending on the Stripe account in use; display currency may differ and is converted via exchange rates.
- **C-006:** Supported currencies are: AED, INR, USD, EUR, GBP, EGP, CAD.
- **C-007:** The default platform commission rate is 5% of order total.
- **C-008:** The service fee (applied to customer orders) is non-refundable on customer-initiated cancellations.
- **C-009:** Customers may cancel orders only up to 24 hours before the earliest event in the order.
- **C-010:** Media assets use Cloudinary in production; local Multer is used for transit only.
- **C-011:** The BullMQ worker process must run in parallel with the main server for background jobs to execute.
- **C-012:** Debug logging for CORS and authentication must not be removed from production.
- **C-013:** "Shadow fields" (featuredImage / featuredImageAsset, images / imageAssets) must coexist during active media migrations.

2.6 Assumptions and Dependencies

- **A-001:** The deployment environment has a Redis instance accessible from both the main server and the worker process.
- **A-002:** The Stripe account is based in India (INR primary) with Stripe Connect used for vendor routing.
- **A-003:** Firebase Authentication is used as a fallback or complement to JWT-based auth; Firebase Admin SDK is initialized at startup.
- **A-004:** All file uploads go through Cloudinary; no permanent local file storage.
- **A-005:** The platform targets the UAE market as primary; multi-currency display is secondary.
- **A-006:** Email delivery uses SMTP via Nodemailer; no transactional email vendor (e.g., SendGrid) is assumed in the base stack.
- **A-007:** The frontend is deployed to Vercel; CORS allows Vercel preview domain regex patterns.

3. Functional Requirements

Requirements are identified by the format `FR-[DOMAIN] - [NNN]`.

3.1 Authentication and Authorization

3.1.1 User Registration

FR-AUTH-001: The system shall allow a new user to register with a first name, last name, email address, and password. The email address must be unique across all user accounts.

FR-AUTH-002: The system shall hash passwords using bcrypt with a salt factor of 10 before persisting to the database. Plaintext passwords must never be stored.

FR-AUTH-003: Upon successful registration, the system shall send an email containing a 6-digit OTP to the user's email address for verification. The OTP shall expire after a configurable duration.

FR-AUTH-004: A user account shall be created with `status` `pending` and `isEmailVerified`: `false` until the email OTP is confirmed.

FR-AUTH-005: The system shall assign the default role of `customer` to all newly registered users unless explicitly specified by an admin.

FR-AUTH-006: The system shall support social login via Google, Facebook, and Apple OAuth providers. A user registered via social login does not require a password.

FR-AUTH-007: The system shall link social provider accounts to existing email-matched user records on first social login, rather than creating duplicate accounts.

3.1.2 Login and Token Management

FR-AUTH-008: The system shall authenticate users by verifying the submitted password against the stored bcrypt hash.

FR-AUTH-009: Upon successful login, the system shall issue a JWT access token signed with HS256. The token payload shall include the user ID, role, and expiry. Clock tolerance for verification shall be 60 seconds.

FR-AUTH-010: The system shall prefer delivering the JWT in an HTTP-only, secure cookie. An `Authorization: Bearer <token>` header shall be accepted as a fallback for non-browser clients.

FR-AUTH-011: The system shall issue a refresh token and store it in the `RefreshToken` collection in MongoDB. The refresh token shall be used to obtain new access tokens without re-authentication.

FR-AUTH-012: The system shall maintain a Redis cache of authenticated user records to avoid redundant database reads on every request. The cache entry shall be invalidated on logout or user update.

FR-AUTH-013: The system shall record login attempts including timestamp, IP address, user agent, and success/failure status in the `loginAttempts` array on the user record.

FR-AUTH-014: The system shall support Firebase UID as an alternative identifier. When a Firebase-authenticated request is received, the system shall verify the Firebase ID token via the Firebase Admin SDK and resolve the matching platform user.

3.1.3 Password Management

FR-AUTH-015: The system shall support password reset via a tokenized link sent to the user's email. The reset token shall expire after a configurable duration.

FR-AUTH-016: The system shall support password reset via an OTP delivered to the user's email, as an alternative to the tokenized link method.

FR-AUTH-017: The system shall support phone number verification via a numeric OTP with configurable expiry.

3.1.4 Two-Factor Authentication

FR-AUTH-018: The system shall support enabling TOTP-based two-factor authentication (2FA) for user accounts. When enabled, a valid TOTP code must be submitted alongside credentials at login.

FR-AUTH-019: The system shall generate and store backup codes for 2FA, allowing account recovery if the TOTP device is unavailable.

3.1.5 Role-Based Access Control

FR-AUTH-020: The system shall enforce RBAC on all authenticated endpoints. The defined roles are: `admin`, `vendor`, `teacher`, `customer`, and `employee`.

FR-AUTH-021: Admin routes (`/admin/*`) shall be accessible only to users with the `admin` role.

FR-AUTH-022: Vendor routes (`/vendors/*`, `/vendor/*`) shall be accessible only to users with the `vendor` role and an associated active vendor profile.

FR-AUTH-023: Teacher routes (`/teachers/*`) shall be accessible only to users with the `teacher` role.

FR-AUTH-024: Checkin routes (`/checkin/*`) shall be accessible to users with the `employee` or `admin` role.

FR-AUTH-025: Public routes (event listings, search, homepage) shall be accessible without authentication.

3.2 Event Management

3.2.1 Event Types

FR-EVT001: The system shall support the following event types: Olympiad, Championship, Competition, Event, Course, Venue, Workshop, Class, Bootcamp, Masterclass.

FR-EVT002: The system shall support the following venue types for each event: Indoor, Outdoor, Online, Offline. Online events must include a `meetingLink`.

3.2.2 Event Creation and Fields

FR-EVT003: A vendor or admin shall be able to create an event with the following required fields: title (max 200 chars), description (rich HTML), category, type, venue type, location (city, address, coordinates), price, currency, age range, and at least one date schedule entry.

FR-EVT004: Events shall support a flexible date schedule with multiple entries. Each schedule entry shall contain: start date, end date, start time, end time, available seats, total seats, price, and optionally unlimited seats (for online events). Time slots within a schedule are also supported.

FR-EVT005: Events shall support educational content fields for Course/Class/Bootcamp/Masterclass types: syllabus (with lessons), subject, topic, intro video URL, skill level (Beginner, Intermediate, Advanced, All Levels), prerequisites, and a linked `teacherId`.

FR-EVT006: Events shall support competition-specific fields: `competitionFormat` (Individual or Team) and `teamSize` for team-format competitions.

FR-EVT007: Events shall support a configurable registration form (`registrationConfig`) with dynamic fields including: text, email, number, phone, textarea, dropdown, checkbox, radio, file, and date field types.

FR-EVT008: Each event shall have an SEO metadata block: SEO title, description, and keywords array.

FR-EVT009: Each event shall support an FAQ list with question-answer pairs.

FR-EVT010: Events shall carry a URL-friendly `slug` field, unique across all events.

FR-EVT011: Events shall support affiliate tracking: `affiliateCode`, `isAffiliateEvent` flag, `externalBookingLink`, click tracking counters, and claim status (unclaimed, claimed, not_claimable).

FR-EVT012: Venue-type events shall support additional fields: operating hours, check-in gates, access rules, facilities, amenities, WiFi credentials, safety features, certifications, insurance info, and capacity.

3.2.3 Event Lifecycle

FR-EVT013: An event created by a vendor shall be placed in `pending` status and must be reviewed and approved by an admin before it becomes publicly visible (`published`).

FR-EVT014: The system shall support event statuses: draft, pending, published, archived, rejected. Status transitions shall be enforced.

FR-EVT015: A vendor or admin shall be able to cancel an event. Cancellation shall: set `cancellationStatus` to `cancelled`, record the reason and timestamp, send notifications to all ticket holders, and initiate refunds for paid orders.

FR-EVT016: The system shall track cancellation notification delivery: total attendees to notify, notified count, and failed count.

FR-EVT017: The system shall support soft deletion of events (`isDeleted: true`, `deletedAt`). Soft-deleted events shall not appear in public listings.

3.2.4 Seat Management

FR-EVT018: The system shall track available seats, sold seats, and reserved seats per schedule entry.

FR-EVT019: When an order is confirmed, the system shall decrement `availableSeats` and increment `soldSeats` for the corresponding schedule entry. If `unlimitedSeats` is `true`, no seat decrement shall occur.

FR-EVT020: Booking attempts for a schedule entry with zero available seats shall be rejected with an appropriate error response.

3.2.5 Ratings and Reviews

FR-EVT021: The system shall maintain aggregated rating data per event: `averageRating`, `reviewCount`, and a `ratingDistribution` object with counts per star level (1–5).

FR-EVT022: The system shall optionally aggregate Google Place ratings (`googleRating`, `googleReviewCount`) and combine them with platform ratings into `combinedRating` and `combinedReviewCount`.

3.3 Booking and Order Management

3.3.1 Order Creation

FR-ORD-001: An authenticated customer shall be able to create an order containing one or more order items. Each order item references an event, a schedule date, a quantity (1–50), unit price, total price, and currency.

FR-ORD-002: The system shall auto-generate a unique order number in the format `GM-{timestamp}-{random}` on first save.

FR-ORD-003: The system shall calculate the order subtotal as the sum of all item total prices. The order total shall equal: `subtotal + tax + serviceFee - discount - couponDiscount`. The total shall never be negative.

FR-ORD-004: Orders shall record a billing address including: first name, last name, email, phone, address, city, state, zip code, and country.

FR-ORD-005: Orders shall support optional participant information per order item. Participants under 18 years old require an emergency contact (name, relationship, phone).

FR-ORD-006: Orders shall support dynamic registration form data captured from the event's `registrationConfig` fields, stored as structured `registrationData` per participant.

FR-ORD-007: The system shall record the order source: `web`, `mobile`, `admin`, or `vendor`.

FR-ORD-008: Orders shall support optional fields: special requests (max 1000 chars), accessibility needs, dietary restrictions, notes (max 500 chars), affiliate code, and coupon code.

3.3.2 Order Status Lifecycle

FR-ORD-009: Orders shall progress through the following statuses: `pending` → `confirmed` (on payment success) → `cancelled` or `refunded`.

FR-ORD-010: Payment status shall be tracked independently from order status with values: `pending`, `paid`, `failed`, `refunded`.

FR-ORD-011: Vendor-facing order status shall be tracked separately: `processing`, `preparing`, `ready`, `completed`, `issue`.

FR-ORD-012: When an order is confirmed (payment succeeds), the system shall automatically trigger ticket generation for all order items.

3.3.3 Cancellation and Refund

FR-ORD-013: A customer may cancel an order only if the order status is `pending` or `confirmed`. For confirmed orders, cancellation is only permitted at least 24 hours before the earliest event date in the order.

FR-ORD-014: The cancellation type shall be recorded: `user_requested`, `event_cancelled`, or `admin_cancelled`.

FR-ORD-015: The refund amount shall be calculated as the event price portion (subtotal minus coupon discount). The service fee shall not be refunded on customer-initiated cancellations.

FR-ORD-016: For event-cancelled or admin-cancelled orders, the system shall process a full refund of the event price.

FR-ORD-017: Refund status shall be tracked: `pending`, `processing`, `completed`, `failed`. Refund transaction IDs shall be stored.

3.3.4 Multi-Currency

FR-ORD-018: Orders shall record both display currency/amount (customer-facing) and charged currency/amount (Stripe-facing). The exchange rate used shall be persisted on the order.

FR-ORD-019: The charged currency shall default to INR for the platform Stripe account. Vendors using Stripe Connect may charge in AED or other supported currencies.

3.3.5 Check-In

FR-ORD-020: An employee or admin shall be able to check in an order. Check-in shall only succeed if the order status is `confirmed` and payment status is `paid`. The check-in timestamp and actor shall be recorded.

3.3.6 Communication Logs

FR-ORD-021: The system shall maintain a communication log per order recording: type (email, SMS, push, call), subject, message body, sent timestamp, and delivery status (`sent`, `delivered`, `failed`, `bounced`).

3.4 Payment Processing

3.4.1 Payment Gateways

FR-PAY-001: The system shall process payments via Stripe as the primary payment gateway. PayPal and Razorpay are enumerated as future gateways but may not be active.

FR-PAY-002: The system shall support the following payment methods: credit card, debit card, digital wallet (Apple Pay, Google Pay, Samsung Pay, PayPal), and bank transfer.

3.4.2 Stripe Integration

FR-PAY-003: The system shall create a Stripe PaymentIntent for each order. The `paymentIntentId` shall be stored on both the Order and Payment records.

FR-PAY-004: The system shall support two Stripe routing modes:

- **Platform routing:** Payment is captured by the platform Stripe account; commission is retained; vendor payout is disbursed separately.
- **Vendor Stripe Connect routing:** Payment flows directly to the vendor's connected Stripe account with the platform retaining an application fee equal to the commission amount.

FR-PAY-005: The `paymentRouting` field on the Order shall record: whether vendor Stripe is used, the vendor Stripe account ID, platform commission amount, vendor payout amount, and Stripe application fee.

FR-PAY-006: The system shall process Stripe webhook events and record each event (event type, event ID, received timestamp, processed flag) in the Payment record's `webhookEvents` array.

3.4.3 Payment Security

FR-PAY-007: The system shall record fraud detection metadata per payment: fraud score (0–100), fraud status (`low_risk`, `medium_risk`, `high_risk`, `blocked`), and security check results for CVV, AVS, and 3D Secure.

FR-PAY-008: The system shall record the last 4 digits and brand of the card used, without storing full card numbers or CVV codes.

3.4.4 Fees

FR-PAY-009: The system shall calculate and record a platform fee and gateway fee per payment. The net amount to the platform shall be `amount - platformFee - gatewayFee`.

FR-PAY-010: The service fee rate shall default to 5% of the order subtotal, configurable per order. The service fee shall be non-refundable on customer-initiated cancellations.

3.4.5 Refunds

FR-PAY-011: The system shall support partial and full refunds. Each refund shall be recorded in the Payment record's `refunds` array with: refund ID, amount, reason, status, and timestamp.

FR-PAY-012: The system shall prevent refunds from exceeding the total payment amount minus previously refunded amounts.

FR-PAY-013: Payment status shall update to `refunded` when the total refunded equals the payment amount, or `partially_refunded` for partial refunds.

3.5 Ticket Management

3.5.1 Ticket Generation

FR-TKT-001: The system shall automatically generate one or more tickets when an order is confirmed. Each ticket corresponds to one event attendee/participant in the order.

FR-TKT-002: Each ticket shall have a globally unique ticket number and a unique QR code string. A QR code image (base64 or URL) shall be generated and stored.

FR-TKT-003: Ticket generation shall be processed asynchronously via a BullMQ job queue to avoid blocking the HTTP request.

FR-TKT-004: Tickets shall carry the following information: ticket number, order ID, user ID, event ID, vendor ID, ticket type, attendee name, attendee email, attendee phone (optional), price, currency, seat number (optional), and seats allocated.

FR-TKT-005: Tickets shall have a validity window: `validFrom` and `validUntil` dates. Tickets past their `validUntil` date shall be automatically expired on next save.

3.5.2 Ticket Statuses

FR-TKT-006: Tickets shall support the following statuses: `active`, `used`, `cancelled`, `refunded`, `expired`, `transferred`.

FR-TKT-007: A ticket may only be marked as `used` if its status is `active` and it has not expired.

FR-TKT-008: A ticket may only be cancelled if it has not been checked in (`used`).

3.5.3 Check-In

FR-TKT-009: An employee or admin shall be able to scan a ticket QR code to check in an attendee. The check-in shall record: check-in time, the employee who performed it, check-in location, and a running scan count.

FR-TKT-010: The system shall reject check-in attempts for tickets with status other than `active`, or that have expired or already been used.

3.5.4 Ticket Transfer

FR-TKT-011: An active, non-expired, non-checked-in ticket shall be transferable to another user. The transfer shall update the ticket's `userId`, set status to `transferred`, and append an entry to the `transferHistory` array with: from user ID, to user ID, transfer timestamp, and optional reason.

3.5.5 Ticket Delivery

FR-TKT-012: Upon successful ticket generation, the system shall send a confirmation email to the attendee's email address containing the ticket details and QR code image.

3.6 Vendor Management

3.6.1 Vendor Profile

FR-VND-001: A user with role `vendor` shall have an associated Vendor profile record. The Vendor profile shall include: business name (max 100 chars), description (max 2000 chars), category, logo, cover image, contact email, phone, contact person, business address, and optional location coordinates.

FR-VND-002: The system shall support vendor business metadata: business hours, social media links, website, tax information (tax ID, VAT number, registration number, business type), and verification documents.

FR-VND-003: Verification documents shall include: business license, tax certificate, and identity document. Each document has a URL, upload timestamp, and verification status (`pending`, `verified`, `unverified`, `rejected`).

FR-VND-004: The system shall maintain cached statistics per vendor: total events, total bookings, total revenue, average rating, total reviews, and last calculated timestamp.

3.6.2 Vendor Verification

FR-VND-005: Vendors shall be created with verification status `unverified`. An admin must review the vendor's documents and change the status to `verified`, `rejected`, or `pending`.

FR-VND-006: Only verified vendors shall be able to switch to custom Stripe payment mode.

FR-VND-007: A vendor may be suspended. Suspended vendors cannot create new events or request payouts. The suspension reason shall be recorded.

3.6.3 Vendor Payment Modes

FR-VND-008: The system shall support two payment modes for vendors:

- **Platform Stripe (`platform_stripe`):** The platform collects payment; deducts commission; vendor requests payout. Default commission rate is 5%. Custom rates can be negotiated and stored with date ranges and admin approval records.
- **Custom Stripe (`custom_stripe`):** The vendor's own Stripe account (via Stripe Connect or manual keys) collects payment directly. No per-transaction commission applies; instead a monthly subscription fee (default 150 AED) is charged.

FR-VND-009: The system shall support Stripe Connect onboarding for vendors. The vendor's `stripeConnectAccountId` and onboarding completion status shall be stored. Stripe Connect capability statuses for card payments and transfers shall be tracked.

FR-VND-010: Manual Stripe secret keys entered by vendors shall be stored with the `select: false` flag to prevent inadvertent API exposure. They must never be returned in standard query responses.

FR-VND-011: The effective commission rate for a vendor shall be determined in this precedence order: subscription mode (0%), active commission agreement rate, custom negotiated rate, default rate.

3.6.4 Vendor Subscription

FR-VND-012: Vendors using Custom Stripe mode shall maintain a subscription with statuses: `active`, `inactive`, `pending`, `expired`, `suspended`, `grace_period`.

FR-VND-013: The system shall record the full subscription payment history per vendor: payment date, amount, period start/end, status, transaction ID, and invoice URL.

FR-VND-014: The subscription `paidUntil` date shall be checked on payment routing. Expired subscriptions shall be flagged and the vendor notified.

3.6.5 Vendor Payout Settings

FR-VND-015: Vendors shall configure payout preferences: schedule (`daily`, `weekly`, `monthly`), minimum payout threshold (default 50 AED), preferred method (`bank_transfer`, `stripe`, `paypal`), and bank account details (IBAN, SWIFT, account number, routing number).

3.7 Teacher Management

3.7.1 Teacher Profile

FR-TCH-001: A user with role `teacher` shall have an associated Teacher profile. The profile shall include: qualifications (degree, institution, year, country, certificate), teaching mode (`online`, `offline`, `hybrid`), subjects, availability hours, social links, bio, and languages spoken.

FR-TCH-002: Teachers shall follow the same verification workflow as vendors: `unverified` → `pending` → `verified` / `rejected`.

FR-TCH-003: Teachers shall have their own payment mode, commission structure, payout settings, and subscription model, mirroring the vendor payment system.

FR-TCH-004: Teachers shall be linkable to events via the event's `teacherId` field. A teacher may be associated with multiple events.

3.7.2 Teacher Payout

FR-TCH-005: The system shall provide separate payout and revenue routes for teachers: `/teachers/payouts` and `/admin/teachers/payouts`.

FR-TCH-006: Teacher revenue shall be tracked separately from vendor revenue in the admin revenue reporting interface.

3.8 Admin Management

3.8.1 User Administration

FR-ADM-001: An admin shall be able to list, search, filter, create, update, suspend, activate, and delete user accounts of any role.

FR-ADM-002: An admin shall be able to view and modify the role of any user.

FR-ADM-003: An admin shall be able to view all login attempts, last login timestamp, and account status for any user.

3.8.2 Event Moderation

FR-ADM-004: An admin shall be able to view all events in any status, including those pending review.

FR-ADM-005: An admin shall be able to approve, reject, archive, feature, or force-publish any event. Approval changes the event status from `pending` to `published`.

FR-ADM-006: An admin shall be able to cancel any event, triggering the full cancellation and refund notification flow.

FR-ADM-007: An admin shall have access to bulk import/export tools for events via the `/admin/bulk-import` routes.

3.8.3 Vendor and Teacher Administration

FR-ADM-008: An admin shall be able to list, search, filter, and update vendor profiles, including changing verification status and suspension status.

FR-ADM-009: An admin shall be able to view a vendor's commission configuration, payment mode, and subscription status, and manually update these fields.

FR-ADM-010: An admin shall be able to create and manage commission agreements for individual vendors, including setting custom rates with date ranges.

FR-ADM-011: An admin shall be able to perform all equivalent operations for teacher profiles.

3.8.4 Payout Administration

FR-ADM-012: An admin shall be able to view all payout requests in any status, filterable by vendor, status, and date range.

FR-ADM-013: An admin shall be able to approve a pending payout, recording the approval timestamp and notes.

FR-ADM-014: An admin shall be able to reject a pending payout with a mandatory rejection reason.

FR-ADM-015: An admin shall be able to mark an approved payout as processing, then as completed (with a transaction/transfer ID) or failed (with a failure reason).

FR-ADM-016: An admin shall be able to cancel a pending or approved payout.

3.8.5 Revenue and Analytics

FR-ADM-017: The admin dashboard shall display aggregated revenue metrics: total revenue, platform commission earned, vendor payouts disbursed, and counts by revenue stream (booking, addon, subscription).

FR-ADM-018: The system shall record a `RevenueTransaction` for each paid order, capturing: order ID, vendor ID, customer ID, total amount, admin commission, vendor payout amount, commission rate, currency, revenue stream, category, and status.

FR-ADM-019: The admin shall be able to view revenue trends over configurable time ranges via analytics endpoints.

FR-ADM-020: The admin shall have access to event-level analytics: views count, booking count, revenue per event.

3.8.6 Commission Configuration

FR-ADM-021: An admin shall be able to create and manage commission configurations with: name, version, status (`active`, `inactive`, `archived`), platform commission defaults (percentage, min/max amounts, currency), and individual rules.

FR-ADM-022: Commission rules shall support three types: `percentage`, `fixed`, `tiered`. Tiered rules shall define amount brackets with individual percentage rates per bracket.

FR-ADM-023: Commission rules shall support recipients: `vendor`, `affiliate`, `referrer`, `platform`.

FR-ADM-024: Commission rules shall support conditions: event categories, min/max order amounts, and vendor tiers.

FR-ADM-025: Only one commission configuration may be marked as `isDefault` at a time. The system shall automatically unset the default flag on all other configurations when a new default is saved.

FR-ADM-026: The system shall support multi-level commission distribution with a configurable number of levels (2–10) and percentage splits per level.

3.8.7 Content and Settings Administration

FR-ADM-027: An admin shall be able to manage CMS content: blog posts, blog categories, SEO content pages, banners, announcement bars, popup notifications, reels, and homepage layout.

FR-ADM-028: An admin shall be able to manage system settings via `/admin/settings` routes, including email settings, payment gateway credentials, and social integration settings.

FR-ADM-029: An admin shall be able to manage event and venue collections (curated groupings).

FR-ADM-030: An admin shall be able to moderate reviews via `/admin/moderation` routes: approve, reject, or delete reviews.

3.9 Commission and Revenue

3.9.1 Commission Calculation

FR-COM-001: The system shall calculate the admin commission amount for each order on save. The calculation shall consider the vendor's payment mode and settings. If the vendor uses Custom Stripe with an active subscription, the commission is 0.

FR-COM-002: The commission rate lookup shall follow this order: vendor's active subscription (0%) → active commission agreement → custom negotiated rate → default platform rate (5%).

FR-COM-003: The commission amount shall be recalculated whenever the order total or commission rate changes.

FR-COM-004: Commission calculations shall not fail the order save on error; fallback to the stored default rate shall apply, and the error shall be logged.

3.9.2 Revenue Transactions

FR-COM-005: When an order is marked as paid, the system shall automatically create a `RevenueTransaction` record linking the order, vendor, customer, total amount, commission, and payout amounts.

FR-COM-006: The `revenueTransactionId` shall be stored on the Order's `adminCommission` sub-document for audit traceability.

FR-COM-007: Revenue transactions shall record the revenue stream (booking, addon, subscription) and commission source (platform_fee, service_fee, addon_fee).

3.9.3 Payout Lifecycle

FR-COM-008: A vendor or teacher shall be able to request a payout for their available balance. The request shall include: amount, currency, payout method, bank details (if applicable), and optional notes.

FR-COM-009: The payout request shall link to the RevenueTransaction IDs it covers, covering the period from periodStartDate to periodEndDate.

FR-COM-010: The payout lifecycle shall be: pending → approved → processing → completed (or failed). An admin may also reject a pending request or cancel a pending/approved request.

FR-COM-011: The system shall support payout methods: bank_transfer, stripe (Stripe transfer), paypal, and manual.

3.10 Content and Marketing

3.10.1 Blog

FR-MKT-001: The system shall support a blog with posts organized into categories. Posts shall have: title, content (rich HTML), featured image, author, tags, SEO metadata, status (draft/published), and view count.

FR-MKT-002: The system shall support blog post comments with a moderation workflow.

3.10.2 Coupons and Discounts

FR-MKT-003: The system shall support discount coupons with: code, discount type (percentage or fixed), discount value, minimum order amount, maximum uses, per-user use limit, expiry date, and applicable event/category restrictions.

FR-MKT-004: At checkout, the system shall validate the coupon code and apply the discount to the order. Invalid, expired, or exhausted coupons shall be rejected with a descriptive error.

3.10.3 Affiliate System

FR-MKT-005: The system shall support affiliate codes. Affiliates shall be trackable via affiliateCode on orders and events.

FR-MKT-006: The system shall track affiliate event clicks: total clicks, unique clicks, and last click timestamp. Affiliate venue clicks shall be tracked separately.

FR-MKT-007: Affiliates shall be able to claim unclaimed affiliate events. The claimStatus shall transition from unclaimed to claimed, recording claimedBy and claimedAt.

3.10.4 Homepage and Banners

FR-MKT-008: The system shall serve aggregated homepage data via a single /homepage endpoint, reducing client-side API calls.

FR-MKT-009: The system shall support configurable banner slots, announcement bars (with scheduling and display rules), and popup notifications (with trigger conditions and frequency controls).

3.10.5 Newsletter

FR-MKT-010: The system shall accept newsletter subscription requests via /newsletter. Subscriber email addresses shall be stored in the NewsletterSubscriber collection.

3.10.6 Search and Discovery

FR-MKT-011: The system shall provide a full-text search endpoint (/search) supporting queries across events, venues, and courses with filtering and sorting.

FR-MKT-012: The system shall allow customers to favorite events. Favorited event IDs shall be stored on the user's favoriteEvents array.

FR-MKT-013: The system shall support event collections (curated groupings), accessible to public at /collections.

3.10.7 SEO

FR-MKT-014: The system shall serve a sitemap and robots.txt at the root API level via /sitemap.xml and /robots.txt.

FR-MKT-015: The system shall support SEO content pages with structured metadata, canonical URLs, and Open Graph fields.

3.10.8 Contact and Partnerships

FR-MKT-016: The system shall accept contact form submissions via `/contact` and store them in the `Contact` collection. An acknowledgement email shall be sent to the submitter.

FR-MKT-017: The system shall accept partnership inquiry submissions via `/partnerships` and store them in the `Partnership` collection.

4. Non-Functional Requirements

4.1 Performance

NFR-PERF-001: API endpoints shall respond within 500ms for 95% of requests under normal load conditions.

NFR-PERF-002: Dashboard and analytics endpoints (heavy aggregation) shall respond within 2 seconds for 95% of requests.

NFR-PERF-003: All read-only database queries shall use Mongoose `.lean()` to bypass document hydration overhead.

NFR-PERF-004: Paginated endpoints shall enforce a maximum page size of 100 items. Unbounded array responses are prohibited.

NFR-PERF-005: Fields not required by the consumer shall be excluded using MongoDB projections (`select()`) to minimize network payload.

NFR-PERF-006: Frequently accessed data (event listings, user profiles post-auth) shall be cached in Redis with appropriate TTL values. Cache invalidation shall occur on relevant write operations.

NFR-PERF-007: Heavy operations (email delivery, QR code generation, ticket PDF rendering, analytics aggregation) shall be offloaded to BullMQ background workers to keep the main event loop free.

NFR-PERF-008: MongoDB queries shall use compound indexes for all common access patterns (user orders by status, event tickets by status, vendor payouts by status/date). All index definitions are documented in the schema files.

NFR-PERF-009: HTTP responses shall be compressed using gzip/Brotli via the `compression` middleware.

4.2 Security

NFR-SEC-001: All HTTP responses shall include security headers via the `helmet` middleware (X-Frame-Options, HSTS, X-Content-Type-Options, CSP, etc.).

NFR-SEC-002: CORS policy shall whitelist only the production frontend domain, Vercel preview domains (matched via regex), and localhost in development. No-origin requests are permitted for GET/HEAD/OPTIONS methods only.

NFR-SEC-003: JWT tokens shall be delivered in HTTP-only, Secure, SameSite cookies to mitigate XSS-based token theft. Authorization header fallback is permitted for non-browser API clients.

NFR-SEC-004: Vendor Stripe secret keys stored in the database shall have the `select: false` Mongoose flag. They must never appear in API response bodies.

NFR-SEC-005: All incoming request bodies, query parameters, and path parameters shall be validated using `express-validator` rules before reaching service logic.

NFR-SEC-006: The system shall apply mongo-sanitize or equivalent to prevent NoSQL injection via user-controlled query operators.

NFR-SEC-007: Rate limiting shall be enforced:

- General API: 100 requests per minute per IP
- Auth endpoints: 10 attempts per 15 minutes per IP
- Dashboard/analytics: 20 requests per minute per IP

NFR-SEC-008: Passwords shall never be logged, stored in plaintext, or returned in API responses.

NFR-SEC-009: Stripe webhook payloads shall be verified using the Stripe-Signature header and the webhook signing secret before processing.

NFR-SEC-010: Fraud detection data (fraud score, AVS/CVV/3DS results) shall be recorded on every payment record for audit and dispute resolution.

NFR-SEC-011: Login attempts (success and failure) shall be logged per user account including IP address and user agent.

NFR-SEC-012: The system shall not expose internal error stack traces or database error details to clients. Generic error messages with error codes shall be returned; full details logged server-side only.

4.3 Scalability

NFR-SCA-001: The service layer shall be stateless; all session state shall reside in Redis or JWT tokens, enabling future horizontal scaling.

NFR-SCA-002: MongoDB connection pooling shall be configured with an appropriate pool size based on the deployment instance (min 10, max 100 connections).

NFR-SCA-003: Background job queues shall use named queues (email, QR, tickets, analytics, notifications) to allow independent worker scaling in future.

NFR-SCA-004: The system shall support graceful shutdown: on SIGTERM, in-progress BullMQ jobs shall be allowed to complete before the worker exits (queue draining).

NFR-SCA-005: The platform's CORS configuration shall support Vercel's dynamic preview domain patterns to allow branch deployments without configuration changes.

4.4 Reliability and Availability

NFR-REL-001: Commission calculation failure shall not fail the order save operation. Fallback to the default commission rate (5%) shall apply and the error logged.

NFR-REL-002: Revenue transaction creation failure shall not fail the payment marking operation. The transaction may be created in a subsequent retry.

NFR-REL-003: Ticket generation failure shall not fail the order confirmation. Ticket generation errors shall be logged and the generation may be retried.

NFR-REL-004: Email delivery shall be queued via BullMQ. Failed email jobs shall be retried with exponential back-off per queue configuration.

NFR-REL-005: The system shall handle unhandled promise rejections and uncaught exceptions with a global error handler, preventing process crashes from individual request failures.

NFR-REL-006: All BullMQ queues shall implement appropriate retry strategies and dead-letter handling for consistently failing jobs.

4.5 Maintainability

NFR-MNT-001: All backend modules shall follow the service-layer pattern: controllers handle HTTP I/O only; services contain all business logic; services are importable independently of Express.

NFR-MNT-002: Custom operational errors shall use the `AppError` class extending native `Error`, carrying an HTTP status code and descriptive message. Empty catch blocks are prohibited.

NFR-MNT-003: Structured logging shall be used for all server-side output. Log entries shall include context (route, user ID, operation) and be formatted for machine parsing.

NFR-MNT-004: Test coverage thresholds shall be maintained: backend services at 70–80% line coverage. Tests must be run before committing code.

NFR-MNT-005: Environment-specific configuration shall be managed via `.env` files. No secrets or configuration values shall be hardcoded.

NFR-MNT-006: "Shadow fields" during data migrations (e.g., `featuredImage + featuredImageAsset`) shall coexist without modification until the migration script completes and is verified.

4.6 Usability

NFR-USA-001: The frontend shall function as a Progressive Web Application (PWA) supporting offline access to cached content and network-first fetching for API data (5-minute cache TTL).

NFR-USA-002: The system shall support internationalization (i18n) via `i18next`. At minimum, English shall be supported; Arabic is a target secondary language for the UAE market.

NFR-USA-003: The API shall return consistent, machine-readable error response bodies with at minimum: `message` string and appropriate HTTP status codes.

NFR-USA-004: All paginated responses shall include metadata: current page, total pages, total items, and items per page.

5. External Interface Requirements

5.1 Stripe

EI-STRIPE-001: The system shall integrate with the Stripe REST API (version 18.4 SDK) for payment intent creation, capture, and refund operations.

EI-STRIPE-002: The system shall support Stripe Connect for routing payments to vendor-connected accounts via `transfer_data` and `application_fee_amount` parameters on `PaymentIntents`.

EI-STRIPE-003: The system shall expose a Stripe webhook endpoint (`/payments/webhook` or equivalent) that verifies the `Stripe-Signature` header using the `stripe.webhooks.constructEvent` method before processing.

EI-STRIPE-004: The system shall handle at minimum the following Stripe webhook events: `payment_intent.succeeded`, `payment_intent.payment_failed`, `charge.refunded`, and `account.updated` (for Connect accounts).

EI-STRIPE-005: The system shall support Stripe Connect account onboarding, storing the connected account ID and capability statuses per vendor.

EI-STRIPE-006: Vendor Stripe secret keys (manual mode) must be validated against the Stripe API before being persisted, recording the last validation timestamp and any validation error.

5.2 Firebase

EI-FIREBASE-001: The system shall initialize the Firebase Admin SDK at server startup using service account credentials from environment variables.

EI-FIREBASE-002: The system shall accept Firebase ID tokens in the `Authorization: Bearer` header as an alternative authentication path. Tokens shall be verified using `admin.auth().verifyIdToken()`.

EI-FIREBASE-003: On successful Firebase token verification, the system shall resolve the corresponding platform user via the `firebaseUid` field or email match, and issue a platform JWT.

EI-FIREBASE-004: The Firebase integration shall function as a fallback authentication mechanism; primary authentication uses the platform JWT system.

5.3 Cloudinary

EI-CLOUDINARY-001: The system shall use Cloudinary as the production media storage and delivery layer for all uploaded images, videos, and documents.

EI-CLOUDINARY-002: The system shall generate Cloudinary transformation URLs via the `CloudinaryUrl` class/utility for responsive image delivery (resizing, format conversion, quality optimization).

EI-CLOUDINARY-003: Upload operations shall use Multer as the in-process multipart parser (transit storage), with files streamed directly to Cloudinary. No permanent local file storage shall be used.

EI-CLOUDINARY-004: The system shall maintain a `MediaAsset` collection in MongoDB cataloguing uploaded assets: Cloudinary public ID, secure URL, resource type, format, dimensions, and size.

EI-CLOUDINARY-005: The `MediaService` abstraction layer shall be used for all media operations to ensure clean separation from the Cloudinary API specifics.

5.4 Redis

EI-REDIS-001: The system shall connect to Redis via the `ioredis` client. The connection must be established before accepting HTTP requests.

EI-REDIS-002: Redis shall serve as the cache store for: authenticated user sessions (post-JWT verification), event listings, vendor profiles, and other frequently read data.

EI-REDIS-003: Redis shall serve as the BullMQ queue broker, backing all job queues: email, QR code, tickets, analytics, and notifications.

EI-REDIS-004: Cache keys shall follow a consistent naming convention (e.g., `user:{id}`, `event:list:{filters}`). TTL values shall be set per cache key type and documented.

EI-REDIS-005: The system shall not use Redis as a primary data store. All business data shall be persisted in MongoDB; Redis is ephemeral.

5.5 BullMQ

EI-BULLMQ-001: The system shall use BullMQ 5.60 backed by Redis for all asynchronous background job processing.

EI-BULLMQ-002: The following named queues shall be maintained:

- **email:** Transactional and notification email delivery
- **qr:** QR code image generation for tickets
- **tickets:** Ticket PDF or record generation
- **analytics:** Analytics event recording and aggregation
- **notifications:** Push and in-app notification delivery

EI-BULLMQ-003: The BullMQ worker process shall run as a separate Node.js process (`npm run dev:worker` in development). It must not be co-located with the main HTTP server process in production.

EI-BULLMQ-004: Job definitions shall include retry configuration. Failed jobs shall be moved to a dead-letter queue after exhausting retries.

EI-BULLMQ-005: The worker process shall implement graceful shutdown: on receiving SIGTERM, no new jobs shall be picked up, and in-progress jobs shall be allowed to complete before process exit.

5.6 Nodemailer / SMTP

EI-EMAIL-001: The system shall send transactional emails via Nodemailer using an SMTP transport configured via environment variables.

EI-EMAIL-002: The `EmailService` shall provide templated email methods for at minimum the following triggers: order confirmation, ticket delivery, order cancellation/refund, password reset, email verification, vendor payout notification, and event cancellation notification to attendees.

EI-EMAIL-003: Email sending shall be queued via BullMQ (the email queue) rather than executed synchronously in the request lifecycle, to prevent SMTP latency from affecting API response times.

6. System Constraints

SC-001 — Single-Core Deployment: The production environment is a KVM1 single-core VPS. All rate limits, connection pool sizes, and concurrency settings are calibrated for single-core capacity. No multi-threaded or cluster-mode Node.js configuration is assumed.

SC-002 — Currency Primary: The platform's primary operating currency is AED (UAE Dirham). The Stripe account charges in INR (Indian Rupee). Multi-currency display is supported but does not change the underlying charged currency. Supported currencies are limited to: AED, INR, USD, EUR, GBP, EGP, CAD.

SC-003 — Cancellation Policy: Customer-initiated order cancellations are only permitted more than 24 hours before the event. Service fees (10% of subtotal) are non-refundable in all customer-initiated cancellation scenarios.

SC-004 — Order Quantity Limit: A single order item may contain a maximum of 50 tickets. This is a hard limit enforced by the Order model schema.

SC-005 — JWT Clock Tolerance: The JWT verification clock tolerance is fixed at 60 seconds and must not be reduced. This accommodates server time-sync drift in the current hosting environment.

SC-006 — Backend TypeScript Strict Mode: TypeScript strict mode is disabled on the backend. Type hints are applied pragmatically but the compiler does not enforce null checks or strict property initialization.

SC-007 — Worker Process Requirement: The BullMQ worker process is a mandatory runtime dependency. Email delivery, QR code generation, and ticket generation will not function without the worker process running. The worker must be started explicitly with `npm run dev:worker`.

SC-008 — Debug Logging in Production: Verbose debug logging for CORS and authentication flows is intentionally present in production and must not be removed. This is required for diagnosing auth issues with the current hosting configuration.

SC-009 — Media Migration Compatibility: The `images` (string array) and `imageAssets` (ObjectId array) fields on Event, and `featuredImage/featuredImageAsset` on Blog, must coexist during the active media migration period. Neither shadow field may be removed before the migration is completed and verified.

SC-010 — Commission Floor: The platform commission rate shall never be set below 0% or above 100% at any level (model validation enforces this range). Vendors on active Custom Stripe subscriptions receive a 0% commission rate; this is not configurable below that floor.

SC-011 — Vendor Stripe Secret Key Security: Vendor Stripe secret keys must never be returned in any API response body. The `Mongoose select: false` flag on the `stripeSecretKey` field is a hard security constraint that must not be overridden without explicit admin action in a controlled context.

SC-012 — No Local File Storage: Permanent file storage on the local server filesystem is prohibited. All media must be uploaded to Cloudinary. Multer local storage is used only as a temporary transit buffer during the upload request and must be cleaned up after Cloudinary upload completes.
